

Convolutional Neural Networks(CNN) Tutorial

Author: Bijo Cheriyan Binoy

Course: MSc Data Science

Student Id: 24084036

GitHub: <https://github.com/bijocbinoy/CNN-MNIST-Project.git>

1. INTRODUCTION

Convolutional Neural Networks (CNN) are types of deep learning algorithms that particularly focus on processing grid-like data, such as images. Unlike conventional neural networks, feature hierarchies in images can be learned automatically through convolutional layers, and this enhances the accuracy of image classification, detection, as well as other applications in the healthcare industry. In this tutorial, we will see how CNNs work, their elements, as well as how one can design a basic CNN for digit classification from the MNIST database. At the end of this tutorial, you will be conversant with CNN design, training, as well as experimentation.

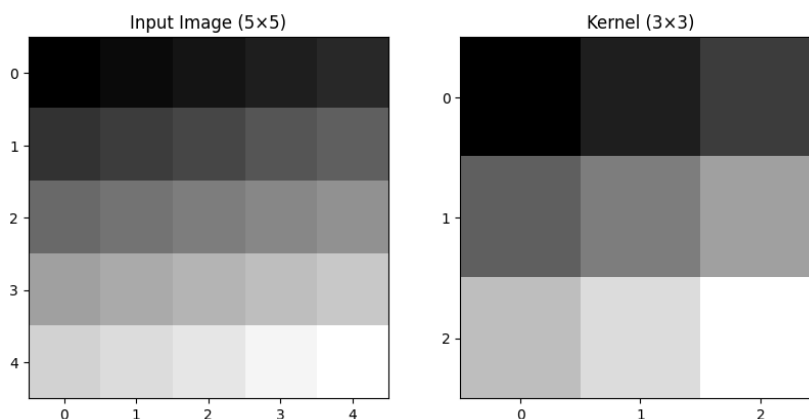
2. CNN Architecture and Key Concepts

CNNs include convolutional layers, activation functions, pooling layers, and fully connected layers. Each part has an important role:

2.1 Convolutional Layer

A convolutional layer scans an image with tiny filters, picking up features as it moves along. At each spot, it adds up what it sees, so you get a feature map that brings out things like edges or textures. How this works really depends on a few hyperparameters filter size, stride, and padding. Filter size changes how much of the image the layer looks at, stride controls how far each step goes, and padding decides if the output keeps its original size or not. Thanks to this setup, the network figures out useful visual details on its own.

Figure 1: Example of a 3×3 filter sliding over a 5×5 input image during convolution.

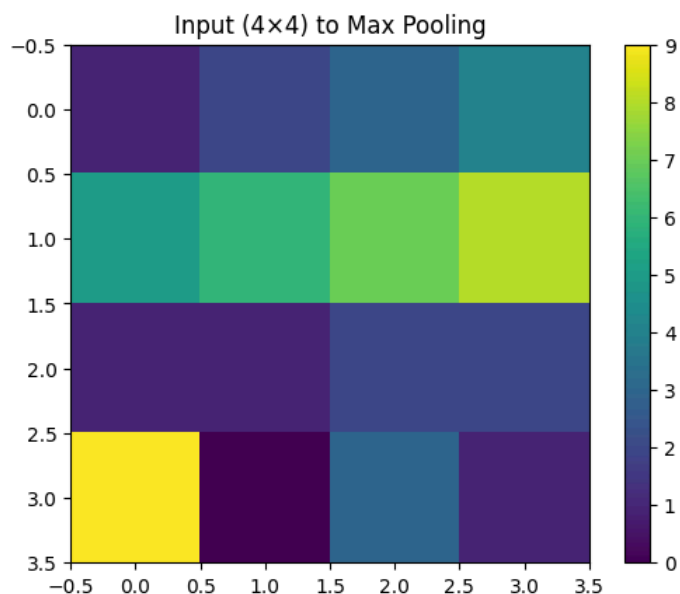


2.2 Activation Function

Here's where things get interesting. The activation function adds non-linearity, which helps the network pick up on all sorts of patterns — not just simple lines. Most people use ReLU. It's pretty straightforward: if the value is less than zero, toss it out; otherwise, keep it. ReLU speeds up training and cuts down on those annoying vanishing gradients that can make networks stall.

2.3 Pooling Layer

Pooling is all about shrinking things down. It takes the big, detailed feature maps and makes them smaller so the network can focus on the really important stuff, and your computer doesn't get overwhelmed. The most common method is max pooling, just grab the biggest value in each patch. Average pooling is another option; it smooths things out by averaging values. For example, in Figure 2 you'd see max pooling in action on a 4×4 feature map with a 2×2 window.



2.4 Fully Connected Layer

Once you've finished with all the filters and pooling, the network flattens everything out into a single vector. Now it connects these features just like a basic neural network would, pulling together all the information to make the final prediction. This is where everything clicks and the network figures out which class your data fits into.

2.5 Optional Layers

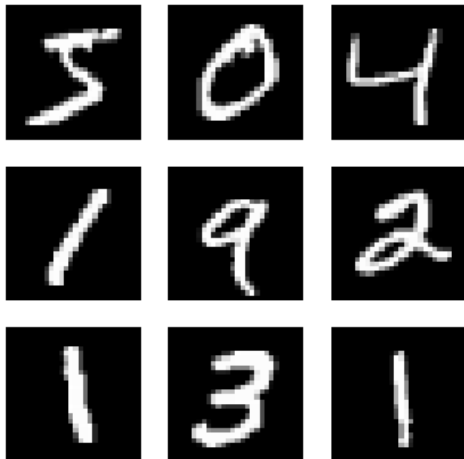
There are layers that are not absolutely required but they are very beneficial. Dropout, for example, in the process of training the network, randomly kills some neurons. This stops the network because of sticking to a single aspect and exclusion. Then there is the batch normalization, it clears the inputs of each layer

hence the training proceeds smoother and the network does not learn in a wavy manner.

3. Dataset: MNIST

The MNIST dataset has 70,000 grayscale images of handwritten digits, from 0 to 9. Each photo is 28 by 28 pixels. People love using it for CNN tutorials because it's simple and fast—you don't need to wait forever for results.

Figure 3: Sample MNIST images (0–9).



4. Hands-on CNN Implementation

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Load dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape(-1,28,28,1)/255.0
x_test = x_test.reshape(-1,28,28,1)/255.0
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Build CNN model
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
```

```

layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])

# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Train model
history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)

# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")

```

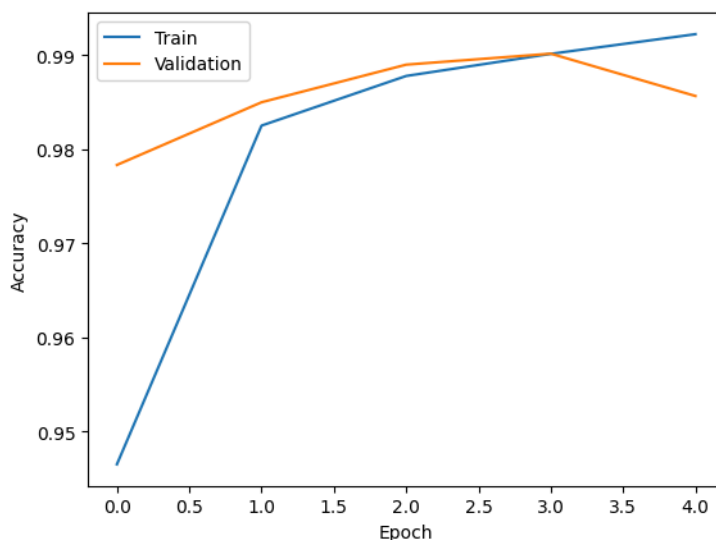
4.1 Visualize Training

```

plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```

Figure 4: Training and validation accuracy over epochs.



4.2 Experimentation

Try switching up the number of filters-bump them up from 32 to 64, or from 64 to 128, and see what happens. Play around with the kernel size too, like going from

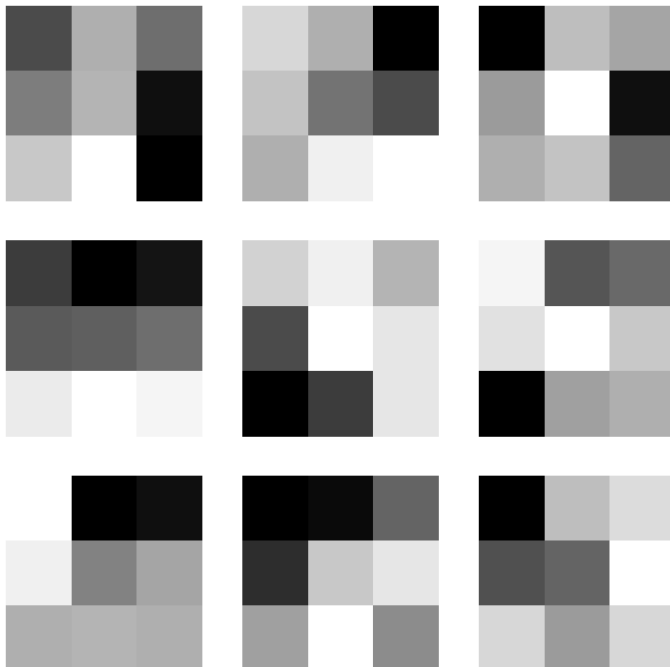
(3,3) to (5,5). Watch how these tweaks change your accuracy and whether overfitting kicks in. Check out Figure 5; it shows a table comparing test accuracy across different setups.

Filters	Kernel Size	Test Accuracy
32-64	3x3	0.987
64-128	3x3	0.992
32-64	5x5	0.984

4.3 Feature Visualization

```
filters, biases = model.layers[0].get_weights()
plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(filters[:, :, :, i].squeeze(), cmap='gray')
    plt.axis('off')
plt.show()
```

Figure 6: Example learned filters from the first convolutional layer.



5. Tips and Best Practices

Keep your input data between 0 and 1, this speeds up training. Don't overcomplicate things at the start. Try a simple model, then make it more complex

if you need to. Set the dropout or data augmentation in order that your network is not that which merely memorizes the data presented to it. Inspect the in-between layers to understand what features the model is actually learning in between. And if you're working with a small dataset, it should not accumulate many layers too many layers do not necessarily translate to better.

6. Conclusion

CNNs are a go-to for image tasks because they pick up on all those spatial patterns in the data, layer by layer. Convolution, pooling, and fully connected layers all work together so your model can actually learn how to tell images apart. Try things out for yourself. When you swap out filters or mess with kernel sizes, you'll see right away how much those choices matter. If you're eager to go deeper, check out popular models like ResNet, VGG, or Inception. They're more advanced, but everything still comes back to the basics you just learned.

7. References

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE.
- Chollet, F. (2018). Deep Learning with Python. Manning Publications.
- MNIST Dataset. <http://yann.lecun.com/exdb/mnist/>
- TensorFlow Keras Documentation: <https://www.tensorflow.org/>